

2. 가상 스레드가 무엇이고 왜 나왔을까?

2.1 스레드 종류

Oracle 공식 문서 : 스레드는 스케줄링될 수 있는 가장 작은 처리 단위이자 `java.lang.Thread` 의 인스턴스이다.

종류

- Platform Thread
- Virtual Thread

2.1.1 Platform Thread

Platform Thread 는 운영체제 스레드를 감싼 얇은 래퍼로 구현된다.

Platform Thread 는 자신의 전체 수명동안 그 운영체제 스레드를 붙잡고 있다.

이 결과 사용 가능한 **Platform Thread** 의 수는 운영체제의 스레드의 수로 제한이 된다.

Platform Thread 는 시작에 **수 천개의 CPU 명령어**가 필요하고 **수 메가바이트 메모리**를 소비하며 서버는 요청 하나당 스레드 하나 배정하는 방식으로 대량의 동시 요청 감당이 어렵다.

2.1.2 Virtual Thread

Virtual Thread : 운영체제 스레드에 묶이지 않는 스레드

가상스레드도 똑같이 `java.lang.Thread` 의 인스턴스지만 특정 운영체제 스레드에 묶여있지 않다.

가상 스레드 안에서 실행 중인 코드가 블로킹 I/O 연산 호출하면 중단된 가상 스레드와 연결돼있던 그 운영체제 스레드는 이제 다른 가상 스레드의 작업 처리할 수 있게 풀려난다.

자바 런타임은 많은 스레드를 시뮬레이션하려고 다수의 가상 스레드를 소수의 운영체제 스레드에 매핑한다.

2.2 언제쓰고 언제쓰지 말아야할까?

2.2.1 적합한 경우

공식문서

- 대부분의 시간을 블로킹된 채로 흔히 I/O 연산 완료를 기다리며 보내는 작업을 실행하는데 적합

예시 : 서버 애플리케이션

리소스를 가져오는 등의 블로킹 I/O 수행하는 수많은 클라이언트의 요청 처리

2.2.2 부적합한 경우

공식문서

- 가상 스레드는 더 빠른 스레드가 아니며 코드를 **Platform Thread** 보다 더 빨리 실행하지 않는다
- 가상 스레드는 **속도**가 아니라 **규모**를 제공하기 위해 존재한다
- 즉 처리량을 늘리는 기술이다

속도 : 더 낮은 지연시간

규모 : 더 높은 처리량

Oracle 채택 가이드 : 가상스레드를 1만개 이상 동시에 다루는 일이 결코 없다면 가상 스레드로 이득 볼 가능성이 낮다

2.2.3 역효과

Platform Thread 는 **희소하며 귀중한 자원**이다 즉, 자원의 희소성을 갖고있다.

귀중한 자원은 관리해야하고 Platform Thread 관리하는 가장 흔한 방법이 **스레드 풀**이다.

풀링은 비싸고 제한된 스레드를 여러 작업이 돌려쓰기 위한 장치이다.

풀에 스레드 갯수 고민하는 이유가 만들 수 있는 스레드 수가 OS 스레드 수로 제한된다

왜 가상 스레드 풀링이 역효과일까?

가상 스레드는 풍부하므로 각각 공유되는 풀링된 자원이 아니라 **하나의 작업을 나타내야**한다.
따라서, "가상 스레드를 몇 개 둘까?" 라는 고민은 무의미하다.

사용자의 이름 정하는데 문자열 몇 개 쓸지 고민하지 않는 것과 마찬가지다

가상 스레드의 수는 언제나 애플리케이션의 동시 작업 수와 같다.

작업이 1만개라면 가상 스레드 또한 1만개이다.

풀링의 본질은 소수의 스레드를 여러 작업이 공유 및 재사용하기 위해서이다.

가상 스레드는 작업 하나당 하나씩 새로 만들면 되는 값 싼 존재이다.

오히려 풀로 묶으면 **작업 = 스레드** 라는 단순한 모델이 깨진다.